

Self Healing Fleet

Anomaly detection e gestione di fault su flotte di robot mobili industriali

Spiegazione del progetto e scelte applicative - materiale di supporto alla relazione

Corso Big Data - Universita' degli Studi Roma Tre - Marcello Tascioni

1. Contesto e problema

Nelle fabbriche e nei magazzini logistici, centinaia di AGV (Automated Guided Vehicles) generano un flusso continuo di dati dai sensori. Un guasto o un errore di traiettoria puo' bloccare parte della produzione. I sistemi di recupero integrati a bordo del singolo robot risolvono errori semplici in tempo reale, ma spesso non gestiscono situazioni complesse come i livelock o la sostituzione automatica in caso di malfunzionamento.

Un sistema centralizzato di anomaly detection, con accesso ai dati continui su traiettorie e stato dei sensori, puo' affrontare queste situazioni: fornire unita' in sostituzione di quelle che presumibilmente si guasteranno, e analizzare i dati storici per riconoscere traiettorie anomale e adattarsi, riducendo i falsi positivi con l'andare della simulazione.

2. Inquadramento Big Data

Il progetto si colloca nel Topic 7 del corso (IoT Stream Analytics): ingestione di stream continui da sensori con analytics real-time e analytics offline (predittiva) sui dati storici. Il principio guida e' che il progetto e' di natura Big Data, non robotica: il robot e' soltanto la sorgente dati, mentre il cuore valutabile e' la pipeline che ingerisce, elabora e sfrutta la telemetria.

Le "V" considerate:

V	Come si manifesta nel progetto
Volume	Molti robot x molti canali x alta frequenza x durata: milioni di record
Velocity	Streaming real-time, detection a bassa latenza
Veracity	Anomaly detection e riduzione progressiva dei falsi positivi
Value	Manutenzione predittiva che previene i fermi di produzione

3. Architettura del sistema

Il flusso dei dati attraversa componenti disaccoppiati, ciascuno con una responsabilita' unica. La sorgente produce telemetria, che viene ingerita da Kafka, elaborata in streaming da Spark, persistita su uno storage colonnare e sfruttata sia dall'analisi offline sia dai due consumatori finali (interrogazione in linguaggio naturale e dashboard).

```
Sorgente (ROS/Gazebo + generatore) -> nodo-ponte con iniezione guasti -> Kafka -> Spark  
Structured Streaming (detection real-time) -> Storage Parquet/Delta -> { analisi offline  
(soglie adattive, previsione) con feedback ; TAG LLM ; Dashboard }
```

4. Scelte applicative e motivazioni

Questa sezione raccoglie le decisioni di progetto con la relativa giustificazione: e' il materiale piu' utile da riportare nella relazione.

Sorgente dati: ROS + Gazebo + TurtleBot3, piu' un generatore sintetico

Perche': ROS con Gazebo e TurtleBot3 fornisce dati di movimento e percezione realistici e conflitti di navigazione emergenti da planner reali. Il generatore sintetico a grafo affianca il braccio reale per lo sweep di scalabilita', perche' Gazebo non raggiunge le decine di migliaia di messaggi/s necessarie a stressare la pipeline.

Alternativa scartata: Un simulatore puramente sintetico sarebbe stato piu' leggero, ma avrebbe rinunciato al realismo e ai livelock emergenti; Gazebo da solo non scala per gli esperimenti di carico.

Modello delle posizioni a grafo

Perche': E' il modo in cui funzionano i sistemi reali di fleet management (roadmap topologica). Rende l'occupazione dei corridoi un dato di prima classe, quindi deadlock e livelock diventano facili sia da rilevare (progresso verso il nodo obiettivo, contesa sugli archi) sia da visualizzare (nodi, archi e robot su un canvas 2D).

Alternativa scartata: Uno spazio continuo puro avrebbe reso il rilevamento delle patologie di traffico piu' oneroso e la visualizzazione web meno immediata.

Canali di salute sintetizzati nel nodo-ponte

Perche': Il TurtleBot3 in simulazione non espone temperatura e corrente motore, e la batteria resta piena. Questi canali - su cui girano detection e previsione - vengono generati nel ponte (valore nominale piu' eventuale firma di guasto), a prescindere da Gazebo.

Iniezione controllata dei guasti

Perche': I guasti di salute sono iniettati come firme parametriche nel tempo (rampa termica, picco di corrente, collasso batteria, sensore bloccato); i guasti comportamentali sono scenari ingegnerizzati (corridoio a corsia singola con task opposti). In entrambi i casi l'evento e' loggato: questo da' un ground truth pulito, indispensabile per misurare precision e recall.

Alternativa scartata: Attendere che le anomalie emergano a caso avrebbe reso le etichette temporali sfumate e il rilevamento non misurabile in modo rigoroso.

Kafka + Spark Structured Streaming

Perche': Sono lo stack canonico e richiesto per lo stream processing, ben documentato e integrato con MLlib. Kafka disaccoppia sorgenti e consumatori e si partiziona per robot_id; Spark Structured Streaming (non le vecchie DStream) esegue la detection real-time con finestre e stato.

Alternativa scartata: Motori piu' recenti (es. Flink) sono validi ma non aggiungono valore didattico qui: al piu' un eventuale braccio di confronto.

Analisi predittiva su serie temporali (offline)

Perche': E' la meta' "offline/predittiva" richiesta dal task. Sui canali di salute storici un modello time-series (ARIMA/Prophet o LSTM) prevede il degrado e stima quando una metrica superera' la soglia critica, ovvero quali robot si guasteranno e con quanto anticipo (lead time). Coincide con la "sostituzione predittiva" prevista fin dal pitch iniziale.

Soglie adattive con feedback

Perche': L'analisi offline ritira le soglie sullo storico e retroagisce sullo streaming, riducendo i falsi positivi nel tempo. E' il punto di originalita' del sistema (Veracity) e produce una curva sperimentale netta da mostrare nella valutazione.

LLM come TAG (text-to-SQL), non come decisore

Perche': I dati sono strutturati e le domande sono aggregazioni esatte ("quanti robot bloccati?"), che si risolvono con una query SQL sullo storico: e' il caso d'uso del TAG (Table Augmented Generation), non

del RAG. Il modello traduce la domanda in SQL, con lo schema intero nel prompt (piccolo, quindi senza schema linking) e una guardia che accetta solo SELECT e ritenta in caso di errore. Un LLM come decisore non aggiungerebbe nulla alle V.

Alternativa scartata: Un vector DB / RAG darebbe risposte per similarita' approssimate invece dei valori esatti; CrewAI sarebbe un framework di agenti superfluo per un task single-shot.

Backend in Node.js con DuckDB embedded

Perche': Il frontend e' comunque una pagina web (canvas 2D). Per il backend si riusa lo script Node gia' esistente che chiama HuggingFace, il push dello stato live in websocket e' idiomatrico in Node, e l'SQL generato viene eseguito sui Parquet con DuckDB in-process. Così lo strato applicativo resta in un solo linguaggio e si sfrutta un asset gia' pronto.

Alternativa scartata: Un backend Python/FastAPI sarebbe altrettanto valido e terrebbe l'intero repo in un linguaggio, ma richiederebbe di riscrivere la chiamata all'LLM.

Dashboard guidata dalla pipeline

Perche': Lo stato live arriva alla pagina dall'output di Spark (topic Kafka fleet_state consumato dal backend e inoltrato in websocket), così la mappa mostra tutti i robot. rosbridge resta un'opzione solo per una vista ROS-ricca dei soli TurtleBot in debug, perché vedrebbe unicamente i robot ROS e non quelli sintetici.

5. Modello dei dati

Lo schema del messaggio di telemetria e' l'unica fonte di verita' a cui tutti i componenti si conformano.

```
telemetry(ts, robot_id, x, y, theta, v_lin, v_ang,
          cmd_v_lin, cmd_v_ang,           # comando vs effettivo
          battery_pct, motor_current, motor_temp, # sintetizzati
          min_obstacle_dist, task_state, # da lidar / stato
          current_edge, goal_node)        # posizione sul grafo
nodes(node_id, x, y, kind) edges(edge_id, from, to, capacity, length)
anomalies(...) injected_faults(...)=ground truth predictions(...)
```

Storage colonnare (Parquet/Delta) nelle directory telemetry/, anomalies/, injected_faults/, predictions/.

6. Tassonomia dei guasti

Tipo	Categoria	Firma / meccanismo
deriva_termica	salute	rampa di temperatura poi plateau
spike_corrente	salute	picco di corrente, resta elevato
batteria_collasso	salute	discesa piu' ripida del nominale
sensore_bloccato	salute	valore congelato (freeze)
deadlock	comportamentale	attesa circolare su segmenti contesi
livelock	comportamentale	due robot che si cedono il passo all'infinito

7. Piano sperimentale (effectiveness + efficiency)

La valutazione copre i due assi richiesti dal professore. Il ground truth prodotto dall'iniezione controllata rende le misure di efficacia rigorose.

Efficiency

- Scalabilità: throughput (msg/s) e latency end-to-end al crescere del carico; individuazione del punto di rottura (con il generatore sintetico).
- Latenza di rilevamento: tempo dall'insorgenza del guasto all'alert.

Effectiveness

- Detection: precision, recall, F1 rispetto a injected_faults; andamento dei falsi positivi nel tempo (effetto delle soglie adattive).
- Previsione: accuratezza del forecast e lead time con cui si anticipa il guasto.
- TAG: execution accuracy (stile benchmark BIRD) su un set di ~20-30 domande di riferimento.

8. Mappatura ai requisiti (Topic 7)

Requisito	Copertura nel progetto
Ingest di stream da sensori	Telemetria TurtleBot -> nodo-ponte -> Kafka
Real-time analytics	Detection in Spark Structured Streaming
Offline analytics (predittiva)	Previsione time-series dei guasti
Kafka / Spark streaming	Ingestion / Structured Streaming
Prediction su time series	ARIMA/Prophet o LSTM
LLM (open source)	TAG text-to-SQL con Qwen via HuggingFace
Esperimenti effectiveness+efficiency	Sezione 7

9. Note per la relazione

- Originalità da valorizzare: soglie adattive con feedback (riduzione dei falsi positivi), modellazione a grafo, interfaccia di interrogazione in linguaggio naturale, sostituzione predittiva.
- Rigore sperimentale: sottolineare che l'iniezione controllata dei guasti fornisce etichette pulite, quindi precision/recall sono misurate su un ground truth affidabile.
- Coerenza con le V: rispondere esplicitamente "quali V" - Volume, Velocity, Veracity (più Value con la predittiva).
- Riproducibilità: codice, script e configurazioni su GitHub; la config di startup e' anche la definizione dell'esperimento.